Axis: A Native Compiled CUDA Training Runtime for Transformer Fine-Tuning

Memory efficiency at 7B scale — engine evaluation with a PyTorch reference comparison

DATE	ACCELERATOR	VERSION	CLASSIFICATION
24 July 2026	NVIDIA A100-80GB	Axis 0.15.1	Public

§ Executive summary

Axis is a native, compiled CUDA training runtime. On a single NVIDIA A100-80GB it reduces GPU memory usage by **17%** relative to an equivalent PyTorch + Transformers + PEFT configuration, while maintaining **comparable training throughput** under identical LoRA fine-tuning conditions. These results demonstrate the effectiveness of Axis's execution architecture — a compiled, fused training step with its own runtime, allocator and autograd — rather than an attempt to maximise raw kernel throughput.

17%
Lower peak GPU memory under matched conditions (23.3 GB vs. 28.2 GB)

0.87×
Relative throughput — comparable, not the optimisation target (3,833 vs. 4,398 tok/s)

3.1e-7
Max numerical drift vs. reference over 30 full training steps

1 Objective

This report evaluates the **Axis** training runtime — a training engine built from first principles rather than assembled from existing frameworks — and characterises its GPU-memory efficiency and training throughput at a representative 7B model scale. A mainstream PyTorch fine-tuning stack is used as a *reference baseline* to place the measurements in context. The goal is not to outperform a mature framework on raw throughput, but to show that a compiled, native execution engine can deliver a materially smaller memory footprint while preserving training correctness.

Full-parameter 7B training does not fit within a single 80 GB accelerator for either system (bf16 weights + gradients + fp32 Adam state ≈ 112 GB); the parameter-efficient LoRA regime is therefore the appropriate single-GPU workload and is the subject of this evaluation.

2 The Axis execution architecture

Axis is not a wrapper around an existing autograd framework. It implements the full training step in a native runtime and executes it without a Python interpreter in the hot loop. The memory and execution characteristics reported here follow directly from these design choices:

COMPONENT	DESIGN
Compiled execution plan	The entire step — embedding, transformer blocks, loss, full backward, and the optimizer update — is lowered once into a single static native plan and executed as one call, or captured as a CUDA graph and replayed.
Native C++/CUDA runtime	Its own runtime owns a dedicated cuBLAS handle and workspace and drives execution directly — no PyTorch, no framework dispatch.
No Python in the training loop	Under graph replay there is no per-operation interpreter overhead; the host issues one launch for the whole step.
Custom autograd	The backward pass is part of the compiled plan rather than a dynamically built tape, eliminating tape bookkeeping and transient allocations.
Fused optimizer	AdamW — with the learning-rate schedule, global-norm gradient clipping and weight decay — runs device-side inside the plan, graph-safe.
Custom allocator	A buffer table addressed by index (not pointer) keeps allocations static and capture-legal, minimising intermediate buffers across the step.
Fused flash attention	Bespoke WMMA tensor-core kernels compute attention forward and backward without materialising the probability matrix; memory stays flat in sequence length.

For low-rank adaptation this architecture is particularly efficient: the frozen base carries no optimizer state and remains in bf16, so intermediate allocations and fp32 optimizer storage are confined to the small adapter set. The compiled step is what makes that saving concrete at runtime, rather than a theoretical property of the method.

3 System under test

HARDWARE	
Accelerator	NVIDIA A100-80GB (Ampere, SXM)
Provider	Modal — one dedicated container per configuration
Isolation	Fresh container per run (cold GPU state, no cross-run caching)

SOFTWARE

CUDA toolkit	12.4.1 (devel image, Ubuntu 22.04)
Python	3.11
Axis	0.15.1 — compiled C++/CUDA engine (own runtime, allocator, autograd, fused flash attention)
Reference stack	PyTorch + Hugging Face Transformers (SDPA/flash) + PEFT (current release)

4 Model and workload

Architecture	Llama-2-7B shape (decoder-only, RoPE, RMSNorm, SwiGLU)
Hidden size / layers	4096 / 32
Attention heads	32 query, 32 key–value (MHA), head dim 128
MLP hidden	11,008
Vocabulary	32,000 (tied embeddings)
Adaptation	LoRA, rank 16, α 32, targets q/k/v/o/gate/up/down (~40M trainable)
Sequence length	2,048
Batch size	1
Precision	bf16 compute, fp32 optimizer state
Optimizer	AdamW (lr 3e-4, β 0.9/0.95, weight decay 0.1)

5 Methodology

- **Matched configuration.** Both systems train the identical LoRA adapter set (same rank and target projections), the same model shape, and the same bf16 + flash-attention path.
- **Measurement.** 2 warm-up steps precede 5 timed steps; wall-clock time is captured with device synchronisation around the timed region. Reported latency is the per-step mean.
- **Throughput.** Tokens per second = (batch × sequence length) ÷ mean step time.
- **Memory.** Peak device memory is read from the driver (`nvidia-smi` , memory-used) and reflects the full process footprint.
- **Isolation.** Each configuration executes in its own fresh container to eliminate allocator and cache carry-over.

6 Results — comparison with the PyTorch reference stack

SYSTEM	LATENCY (MS/STEP)	THROUGHPUT (TOK/S)	PEAK MEMORY	TRAINABLE
Axis (native compiled engine)	534	3,833	**23.3 GB**	40M
PyTorch reference (HF Transformers + PEFT)	466	4,398	28.2 GB	40M

Table 1. Single-A100 7B LoRA fine-tuning. Lower latency and memory are better; higher throughput is better.

FIGURE 1 — PEAK GPU MEMORY (LOWER IS BETTER)

FIGURE 2 — TRAINING THROUGHPUT (HIGHER IS BETTER)

7 Why this matters

The 4.9 GB reclaimed by Axis is not a micro-optimisation — it changes what is trainable on a given device. The same optimization algorithm, precision and adapter configuration can be applied while redirecting that headroom to **longer context windows**, **larger micro-batch sizes** (improving accelerator utilisation), or **fine-tuning on smaller-memory GPUs**. Because the saving is structural — the frozen base holds no optimizer state and stays in bf16 — it is expected to persist across model scales, and is directionally consistent with the 1B-scale measurement (Axis ~38% lower LoRA memory).

8 Correctness and numerical validation

Memory efficiency is only meaningful if training remains correct. The following properties are established by the Axis validation suite, which gates every engine capability against a reference implementation. They characterise the engine's numerical fidelity independently of the performance measurement in §6.

PROPERTY	RESULT	HOW VERIFIED
Numerical parity	max rel. drift 3.1×10^{-7}	vs. reference over 30 full training steps, through the optimizer
Gradient correctness	bit-tight	LoRA adapter gradients vs. reference
Numerical stability	0 NaN, converges	500-step CUDA-graph soak, exact bias correction
Shape robustness	32 / 32 pass	fuzzed depth/width/heads/kv/vocab/tied/LoRA configs
Distributed correctness	bit-identical	multi-GPU + ZeRO-1 vs. replicated data-parallel
Checkpoint / interop fidelity	max $\lvert\Delta\rvert \approx 3\times10^{-6}$	loads Hugging Face Llama checkpoints; reproduces logits

Table 2. Correctness is preserved: the memory savings do not trade off numerical fidelity.

9 Analysis

Memory (the objective). Axis compiles the entire training step into a native CUDA execution plan, minimising intermediate allocations and confining fp32 optimizer state to the ~40M adapters, for a 4.9 GB (17%) reduction in peak footprint. This is a property of the execution architecture, not a tuning artefact.

Throughput (not the objective). The PyTorch reference is ~15% faster per step, reflecting years of hand-tuned CUDA kernels and deep PEFT integration. Axis reaches comparable throughput through CUDA-graph replay and fused flash attention; closing the remaining gap is an ongoing kernel-optimisation effort, not a goal of this release.

10 Scope and limitations

- **Optimisation priority.** Axis prioritises execution efficiency and memory footprint over maximum kernel throughput in the current release; throughput optimisation (kernel autotuning, scheduling) is ongoing.
- **Workload.** Results characterise LoRA fine-tuning only. Full-parameter 7B training exceeds a single 80 GB accelerator for both systems and requires multi-GPU sharding (ZeRO / FSDP), evaluated separately.
- **Regime.** Measurements use a fixed sequence length (2,048) and batch size (1); throughput scales with both and should be re-measured for other regimes.
- **Versions.** Reference-stack library versions track the current public release at the report date and are not pinned.

11 Positioning and future work

Axis is a training runtime, and its natural reference set extends beyond a single framework. Planned evaluations position Axis within the broader landscape of training systems rather than against one baseline: memory and sharding systems (**DeepSpeed ZeRO**, **PyTorch FSDP**, **Megatron-LM**), attention and kernel

libraries (**xFormers**), and alternative execution models (**JAX / XLA**, **tinygrad**). Alongside this, full-parameter multi-GPU training and end-to-end convergence on real datasets are the next measurements on the roadmap.

12 Reproducibility

The benchmark harness and this report are published in the Axis repository. To reproduce on an A100-80GB:

```
modal run engine/modal_7b_bench.py
```

Source: `github.com/Zyora-Dev/axis` · Package: `pip install axis-zyora`